ETAS RTA-BSW v12.7.0

RTA-Hwd (Can) Reference Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Hwd (Can) Reference Guide R01 EN – 05.2025

Contents

1 Can Transceiver 4
1.1 Introduction 4
1.2 Supported Features 5
1.3 Extensions 7
1.4 Limitations 13
1.5 Exclusions 14
1.6 Deviations 15
1.7 API Definition 16
1.8 Configuration Advice 17
1.9 Integration Advice 21
1.10 Hardware Requirements 23
2 Glossary 24

1 Can Transceiver

Module name	CanTrcv
Package	RTA-HWD (Can)
AUTOSAR Module ID	70
AUTOSAR Version	23-11

1.1 Introduction

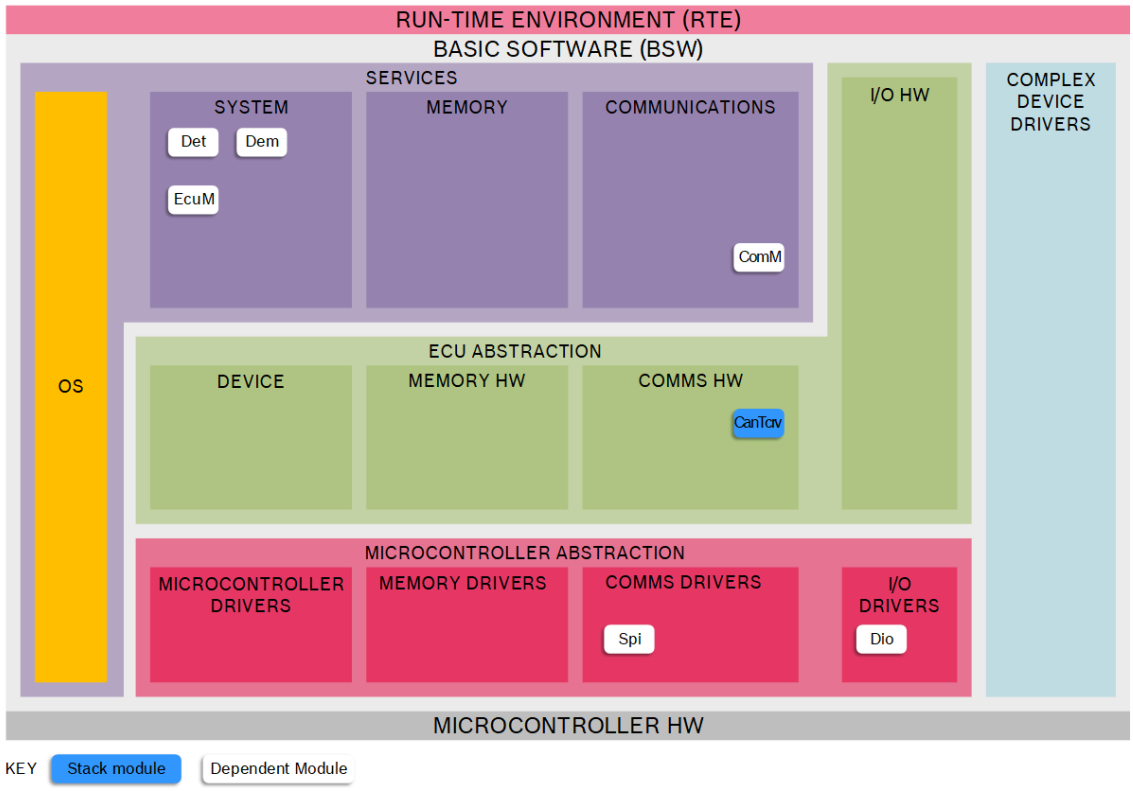

Figure 1.1.CanTrcv Stack Architecture

The Can Transceiver Driver module (CanTrcv) abstracts compatible transceiver hardware devices to a standardized interface which can then be used by the upper layer modules such as CanIf, EcuM or application software. The diagram above shows the position of CanTrcv in the AUTOSAR layered architecture.

The main purpose of the CanTrcv module is as follows:

- To change the state of the transceiver.
- To read out current state of transceiver.
- To detect errors on the CAN bus.
- To detect different wakeup events with wakeup reasons.

CanTrcv implements a separate state machine for each compatible transceiver

device on a Can network. The following AUTOSAR-specified states are recognised:

- POWER_ON
- NOT_ACTIVE
- ACTIVE
- CANTRCV_TRCVMODE_NORMAL
- CANTRCV_TRCVMODE_STANDBY
- CANTRCV_TRCVMODE_SLEEP

For a full description of the behaviours associated with each state, please refer to the document AUTOSAR_SWS_CANTransceiverDriver.pdf

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

> **NOTE**
>
> The functionality provided by CanTrcv depends on the features supported by each hardware transceiver device. For example, if a transceiver does not support wakeup detection, or the internal state machine design does not recognise a particular state, this functionality cannot be provided by the CanTrcv module. You should consult the vendor's hardware-specific documentation for each type of transceiver hardware that you wish to use with CanTcrv.

1.2 Supported Features

1.2.1 Supported Hardware Devices

CanTrcv supports the following hardware devices:

Manufacturer	Device Name
NXP Semiconductors	TJA1040
NXP Semiconductors	TJA1041
NXP Semiconductors	TJA1042 (TCAN1042, TJA1042BT)
NXP Semiconductors	TJA1442
NXP Semiconductors	TJA1043 (TJA1443ATK, TJA1463ATK, TCAN1043A-Q1, TJA1463ATK, TCAN1043A-Q1, TCAN1463-Q1)
NXP Semiconductors	TJA1044GT (TJA1442ATK, TJA1462ATK, TJA1462, TCAN1044A-Q1, TCAN1462-Q1)
NXP Semiconductors	TJA1046TK
NXP Semiconductors	TJA1048

NXP Semiconductors	TJA1145 (TJA1145AT)
NXP Semiconductors	TJA1463 (TCAN1463(TI))
Infineon Technologies	TLE6250G
Infineon Technologies	TLE6250GV33
Infineon Technologies	TLE6251
Infineon Technologies	TLE8250GVIO
Infineon Technologies	TLE9255
Texas Instruments	TCAN1145
Bosch	CY327
Bosch	CS9xx
Microchip	ATA6570
Microchip	ATA6571
Microchip	ATA6572

Other devices may be supported if they have a specific Complex Device Driver (CDD) interface, see the notes on the Extended CDD CanTrcv in the Configuration Advice.

1.2.2 Wakeup Support

CanTrcv supports wakeup for all the hardware devices listed above.

Other devices may be supported if they have a specific Complex Device Driver (CDD) interface.

CanTrcv provides API functions to support wakeup if the CanTrcvWakeUpSupport configuration parameter is set to CANTRCV_WAKEUP_BY_POLLING, and if the hardware device is present in the list above.

1.2.3 Operation Mode Management

`CanTrcv_SetOpMode()` is called by the CanIf module to switch the operation mode (a sub-state of the ACTIVE state) for a transceiver device via DIO or SPI. There are 3 supported operation modes as follows:

- CANTRCV_TRCVMODE_NORMAL - Full bus communication, if the transceiver hardware controls the ECU power supply and the ECU is fully powered. The CanTrcv module detects no further wake up information.
- CANTRCV_TRCVMODE_STANDBY - No communication is possible. The ECU is still powered if the transceiver hardware controls the ECU power supply. A transition to CANTRCV_TRCVMODE_SLEEP is only valid from this mode. A wakeup by bus or by a local wakeup event is possible.
- CANTRCV_TRCVMODE_SLEEP - No communication is possible. The ECU may be powered or unpowered depending on the behaviour of the power supply. A wakeup by bus or by a local wakeup event is possible.

1.2.4 Partial Networking

When some nodes in a Can network are in low power mode (while others are actively communicating), the low-power nodes can be woken up by predefined wakeup frames (WUF). This feature is enabled by setting configuration parameter `CanTrcvHwPnSupport` to TRUE. Only transceivers which support selective wakeup functionality can be used with this feature.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvDioAccess/CanTrcvDioChannelAccess/CanTrcvRbDio-HardwareInterfaceType

Short Name	CanTrcvRbDioHardwareInterfaceType
Description	This Rb parameter defines functionality of DIO pin. The trcv-software uses this parameter to know particular functionality of the referenced Dio Pin connected to the Trcv-HW.
Introduction	DIO pins of tranceiver are configured with Enable, Standby, ErrorPin, WakeupSignalisation or Inhibhit
Multiplicity	1..1
Default Value	-
Enumeration Literals	Enable, Standby, ErrorPin, WakeupSignalisation, Inhibit
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.2 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvDioAccess/CanTrcvDioChannelAccess/CanTrcvRbHard-warePinInvert

Short Name	CanTrcvRbHardwarePinInvert
Description	Parameter depends on hardware: Describes, if a inverter is between transceiver pin and micro controller. If the transceiver is directly connected to the Controller, 'false' should be configured. If there is an inverter between transceiver and micro controller, 'yes' should be configured.
Introduction	false: No inverter between Micro Controller and transceiver true : Inverter between Micro Controller and transceiver
Multiplicity	0..1

Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.3 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvSpiAccess/CanTrcvRbToggleRxD

Short Name	CanTrcvRbToggleRxD
Description	false: The RxD Pin will be constant low true The RxD Pin will toggle between low and high This param is only applicable for TLE9255 CanTrcv
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.4 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvSpiAccess/CanTrcvRbWPFE

Short Name	CanTrcvRbWPFE
Description	false: falling-edge detection on WAKE pin disabled true falling-edge detection on WAKE pin enabled This param is placed here as Tja1145param container is removed in resturcutured code
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.5 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvSpiAccess/CanTrcvRbWPRE

Short Name	CanTrcvRbWPRE
Description	false: rising-edge detection on WAKE pin disabled true rising-edge detection on WAKE pin enabled This param is placed here as Tja1145param container is removed in resturcutured code
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.6 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvDemEventParameterRefs/CANTRCV_RB_E_UNEXPECTED_MODE_PROP

Short Name	CANTRCV_RB_E_UNEXPECTED_MODE_PROP
Description	Reference to the DemEventParameter which shall be issued when the unexpected mode propogation is detcted in Transceiver HW.

Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

1.3.1.7 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvDemEvent-ParameterRefs/CanTrcvRbDynCreate_DemEvents

Short Name	CanTrcvRbDynCreate_DemEvents
Long Name	Dynamically create DemEvent
Description	Selects the way how the example DemEvent is created. If not set or set to Disabled, no such DemEvent is automatically created. If the dynamic creation of this DemEvent is enabled, the reference to this DemEvent (in parameter rba_Example_DemEventRef) is also automatically created and shall therefore not be set manually.
Multiplicity	0..1
Default Value	Disabled
Enumeration Literals	Disabled, Standard
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.8 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvPinWakeup-SourceRef

Short Name	CanTrcvPinWakeupSourceRef
Description	Symbolic name reference to specify the wakeup sources that should be used in the calls to EcuM_SetWakeupEvent for wakeup by PIN.
Introduction	This reference is mandatory if the HW supports PIN wakeup
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

1.3.1.9 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCdd-CheckWakeupApiUsed

Short Name	CanTrcvRbCddCheckWakeupApiUsed
Description	To be enabled if a CDD Check Wakeup API is available for this channel
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.10 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddGet-

BusWakeUpReasonApiUsedCanTrcvRbCddGetBusWakeUpReasonApiUsed

Short Name	CanTrcvRbCddGetBusWakeUpReasonApiUsed
Description	To be enabled if a CDD Get Bus Wakeup reason API is available for this channel
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.11 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddInitApiUsed

Short Name	CanTrcvRbCddInitApiUsed
Description	To be enabled if a CDD Init API is available for this channel
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.12 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddMainApiUsed

Short Name	CanTrcvRbCddMainApiUsed
Description	To be enabled if a CDD Main function API is available for this channel
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.13 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddModeChangeDelay

Short Name	CanTrcvRbCddModeChangeDelay
Description	This parameter need to be configured only if unexpected Mode Propogation monitoring for Transceiver is enabled. Here user need to specify the minimum delay required to set the requested mode of transcier. User have to enter the number in the multiples of main fucntion duration. For example if Main funtion is scheduled in 10ms task and minimum time required to change the transceiver mode is 20ms then configure this parameter as 2 .
Multiplicity	0..1
Value Range	1..255

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.14 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddName

Short Name	CanTrcvRbCddName
Description	Only used if CDD is choosen as HardwareType. A separate freetext field which is used to generate function calls out of CanTrcv into the CDD and to include the corresponding .h file. E:G. if "rba_IoExtCy321" is configured, the CanTrcv code generator generates function calls "rba_IoExtCy321_CanTrcvSetOpMode", "rba_IoExtCy321_CanTrcvGetOpMode", "rba_IoExtCy321_CanTrcvGetBusWuReason", "rba_IoExtCy321_CanTrcvSetWakeupMode" and includes the file rba_IoExtCy321.h.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

1.3.1.15 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbCddSetWakeupModeApiUsed

Short Name	CanTrcvRbCddSetWakeupModeApiUsed
Description	To be enabled if a CDD Set Wakeup mode API is available for this channel
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.16 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbHardwareType

Short Name	CanTrcvRbHardwareType
Description	Selects the Type of the Cantrcv Hardware used The below list provides the similar devices which can be selected from the drop down list in the configuration. TJA1042 ——- TCAN1042/TJA1042BT TJA1043 ——- TJA1443/TJA1443ATK/TJA1463ATK/TCAN1043A-Q1/TCAN1463-Q1 TJA1044GT ——- TJA1442/TJA1442ATK/TJA1462/TCAN1462-Q1/TJA1462ATK/TCAN1044A-Q1/TJA1044CT/TJA1044BT/TCAN1044AVDRBRQ1 TJA1145 ——- TJA1145AT
Multiplicity	1..1

Default Value	-
Enumeration Literals	TJA1040, TJA1041, TJA1042, TJA1044GT, TJA1043, TJA1463, TJA1046TK, TJA1048, TJA1145, TLE9255, TCAN1145, ATA6570, ATA6571, ATA6572, TLE6250G, TLE6250GV33, TLE6251, TLE8250GVIO, CDD, CY327, CS9XX
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.17 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbSPIParam

Short Name	CanTrcvRbSPIParam
Description	Container for the configuration of specific parameters in SPI transceivers.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.18 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvRbSPI-Param/CanTrcvRbCanFDTolerance

Short Name	CanTrcvRbCanFDTolerance
Description	TJA1145 or TCAN1145 CAN FD tolerance enabled (true) or disabled (false). False- Can FD frames will be detected as error frames. True- Can FD frames are ignored.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.19 CanTrcv/CanTrcvGeneral/CanTrcvRbCddTrcvIdUsed

Short Name	CanTrcvRbCddTrcvIdUsed
Description	Enable/disable CanTrcv Channel Id being passed for cdd
Introduction	* true: Channel Id to be passed * false: Channel Id not passed.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.20 CanTrcv/CanTrcvGeneral/CanTrcvRbGetError

Short Name	CanTrcvRbGetError
Description	RB Specific API to read Error pin/Status register
Introduction	TRUE = Is used. FALSE = Is not used.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.21 CanTrcv/CanTrcvGeneral/CanTrcvRbOpModeImmediate

Short Name	CanTrcvRbOpModeImmediate
Description	Enable/disable RB specific CanTrcv_SetOpMode_Immediate, CanTrcv_GetOpMode_Immediate APIs
Introduction	TRUE = Is used. FALSE = Is not used.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.2 Additional Services

The following API functions are available in version v12.7.0 in addition to those defined in AR v22-11

`CanTrcv_SetOpMode_Immediate`	Set the transceiver mode (Only for relevant for SPI-TJA1145 transceivers)
`CanTrcv_Rb_GetError`	Read out error bits or to read out the actual value of the Error pin

1.3.2.1 CanTrcv_SetOpMode_Immediate

Syntax	`CanTrcv_SetOpMode_Immediate`
Parameters (in)	`Transceiver` (uint 8) The ID of the transceiver to operate on.
Parameters (out)	`OpMode` A pointer to the `CanTcrv_TrcvModeType` containing the current operating mode of the specified transceiver.
Description	Set the transceiver mode (only relevant for SPI-TJA1145 transceivers).

1.3.2.2 CanTrcv_Rb_GetError

Syntax	`CanTrcv_Rb_GetError`
Parameters (in)	`Transceiver` (uint 8) The ID of the transceiver to operate on.
Parameters (out)	`Error_pu32` (uint 32) The error bit mask (or 0 for no error). For DIO transceivers, returns the current actual pin value.
Description	Returns the value of the transceiver's Error pin (or undefined pin value if no Error pin is configured).

1.4 Limitations

1.4.1 Dem Events

Dem Events are only checked in CanTrcv_Init for Dio transceivers (i.e. not in

CDD-transceivers where CanTrcv works as a wrapper to specific CDD components, such as with the CY327 or CS9xx). Dem Events are not checked in CanTrcv_GetOpMode, CanTrcv_SetOpMode, CanTrcv_GetBusWuReason or CanTrcv_SetWakeupMode.

1.4.2 Bus Error Detection

Bus error detection is not supported in this release, even if the transceiver hardware is able to detect bus failures.

1.4.3 SPI Support

CanTrcv currently only works with the External Buffer mode of SPI (see Integration Advice for details). Internal Buffer mode is not currently supported.

1.4.4 Undervoltage recovery

This mechanism is not supported.

1.5 Exclusions

API functions not currently implemented:

- `CanTrcv_MainFunctionDiagnostics`
- `CanTrcv_DeInit`
- `CanTrcv_GetTrcvSystemData`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvDioAccess/CanTrcvDioChannelAccess/CanTrcvHardwareInterfaceName
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess/CanTrcvSpiAccess/CanTrcvSpiSequence/CanTrcvSpiSequenceName
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvChannelEcucPartitionRef
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvDemEventParameterRefs/CANTRCV_E_BUS_ERROR
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvIcuChannelRef
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvSyserrWakeupSourceRef
- CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanXLTrcvChannel/CanXLTrcvEthEcucPartitionRef
- CanTrcv/CanTrcvGeneral/CanTrcvEcucPartitionRef
- CanTrcv/CanTrcvGeneral/CanTrcvIndex
- CanTrcv/CanTrcvGeneral/CanTrcvMainFunctionDiagnosticsPeriod
- CanTrcv/CanTrcvGeneral/CanTrcvTimerType

- CanTrcv/CanTrcvGeneral/CanTrcvWaitTime

1.6 Deviations

1.6.1 Dem Events

Dem Events are only checked in function `CanTrcv_Init` for DIO transceivers. They are not checked in `CanTrcv_GetOpMode`, `CanTrcv_SetOpMode`, `CanTrcv_GetBusWuReason` or `CanTrcv_SetWakeupMode`.

A Dem Event is only checked in `CanTrcv_ModeChangePropogationDem` for its mode change.

1.6.2 Det Events

Error Code	Value in Hex	Support status	AUTOSAR
CANTRCV_E_INVALID_TRANSCEIVER	0x01	Supported	Yes
CANTRCV_E_PARAM_POINTER	0x02	Supported	Yes
CANTRCV_E_UNINIT	0x11	Supported	Yes
CANTRCV_E_PARAM_TRCV_WAKEUP_MODE	0x23	Supported	Yes
CANTRCV_E_PARAM_TRCV_OPMODE	0x24	Supported	Yes
CANTRCV_E_PARAM_TRCV_WRONG_CONF	0x28	Supported	No
CANTRCV_E_PARAM_TRCV_WAKEUP_MODE	0x23	Supported	Yes
CANTRCV_E_SPI	0x31	Supported	No
CANTRCV_E_TRCV_NOT_STANDBY	0x21	Supported	Yes
CANTRCV_E_TRCV_NOT_NORMAL	0x22	Not Supported	Yes
CANTRCV_E_BAUDRATE_NOT_SUPPORTED	0x25	Not supported	Yes
CANTRCV_E_NO_TRCV_CONTROL	0x26	Supported	Yes
CANTRCV_E_INIT_FAILED	0x27	Not supported	Yes

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.2.1 CanTrcv/CanTrcvConfigSet

[RTA-BSW] Description	This container contains the configuration parameters and sub containers of the AUTOSAR WdgM module.
[AUTOSAR] Description	This container contains the configuration parameters and sub containers of the AUTOSAR CanTrcv module.

1.6.2.2 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvAccess

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.2.3 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanTrcvMaxBaudrate

[RTA-BSW] Maximum Value	12000
[AUTOSAR] Maximum Value	20000

1.6.2.4 CanTrcv/CanTrcvConfigSet/CanTrcvChannel/CanXLTrcvChannel

[RTA-BSW] Upper Multiplicity	*
[AUTOSAR] Upper Multiplicity	1

1.6.2.5 CanTrcv/CanTrcvConfigSet/CanTrcvSPICommRetries

[RTA-BSW] Description	Indicates the maximum number of communication retries in case of a failed SPI communication (applies both to timed out communication and to er-rors/NACK in the response data).
[AUTOSAR] Description	Indicates the maximum number of communication retries in case of a failed SPI communication (applies both to timed out communication and to errors/NACK in the response data).
[RTA-BSW] Introduction	If configured value is 0, no retry is allowed (communication is expected to succeed at first try).
[AUTOSAR] Introduction	If configured value is '0', no retry is allowed (communication is expected to succeed at first try).
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.2.6 CanTrcv/CanTrcvConfigSet/CanTrcvSPICommTimeout

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.7 API Definition

`CanTrcv_Init`	Called by EcuM or BswM to initialize the module
`CanTrcv_SetOpMode`	Used to set the transceiver mode
`CanTrcv_GetOpMode`	Returns the actual state of the transceiver
`CanTrcv_GetBusWuReason`	Returns the wakeup reason
`CanTrcv_GetVersionInfo`	Returns the version information of this module
`CanTrcv_SetWakeupMode`	Sets the wakeup event for a transceiver according to the value of `TrcvWakeupMode`

`CanTrcv_CheckWakeup`	Notifies the calling function if a wakeup is detected (or not detected) for a transceiver
`CanTrcv_CheckWakeFlag`	Checks the status of the wakeup flag from the transceiver hardware
`CanTrcv_SetPNActivationState`	Configures the wake-up of the transceiver for CANTRCV_TRCVMODE_STANDBY and CANTRCV_TRCVMODE_SLEEP states. The transceiver can be woken up by a remote wake-up pattern (standard wakeup) or by the configured remote wakeup frame
`CanTrcv_ClearTrcvWufFlag`	Clears the WUF flag in the transceiver hardware
`CanTrcv_ReadTrcvTimeoutFlag`	Reads the status of the timeout flag from the transceiver hardware
`CanTrcv_ClearTrcvTimeoutFlag`	Clears the status of the timeout flag in the transceiver hardware
`CanTrcv_ReadTrcvSilenceFlag`	Reads the status of the silence flag from the transceiver hardware
`CanTrcv_MainFunction`	Scheduled function of the CanTrcv module

1.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

1.8.1 Transceiver Configuration

Transceivers are identified by a unique ID, which is set on a per-channel basis using the configuration parameters contained in `CanTrcvChannel`. Therefore, you should configure multi-channel transceivers with a separate set of parameters for each channel (see Multi-Channel Transceivers below).

For transceivers which use the DIO interface, configure the DIO port for the port pin which is connected to the transceiver's Enable, Standby or Error pin. This information will be available in the vendor's ECU schematic. These settings are made under the `CanTrcvDioAccess` parameters.

For transceivers which use the SPI interface, configure the settings under the `CanTrcvSpiAccess` parameters. It is recommended that you configure this type of transceiver for asynchronous access (by setting `CanTrcvSpiAccessSynchronous` to FALSE). Note that SPI parameters are not forwarded and must be configured manually. These parameters are not mandatory and depend on the MCAL used.

1.8.1.1 Multi-Channel Transceivers

Some transceivers have multiple channels, and the name of these transceivers may not be found directly in the transceiver HW list. You should refer to the transceiver's user manual to identify the names of individual transceiver channels, which will need to be provided during configuration.

For example, the TJA1901 is actually: TJA1046 + TJA1029 (2 x TJA1044GT + TJA1029).

TJA1901 will not be available as a name in the transceiver list, but TJA1046 and TJA1029 will be available, and these should be selected instead of TJA1901.

1.8.2 Dependency of the APIs within CanTrcv

When the CanTrcv is requested to enter the sleep state by the application, the transceiver hardware first enters Go-To-Sleep Mode and waits for the (t_h) hold time (50 micro second) and then goes into sleep mode. If within this (t_h) hold time a bus activity is detected by the transceiver hardware it moves into Standby Mode / Wake up flag set;

The SW is not aware of this transition and thinks that its in the sleep state. As the sleep mode request is handled by the application SW, they also have to monitor CanTrcv_Rb_GetError() to identify if the HW has undergone this transition before requesting Sleep mode again. This is applicable for transceivers such as TJA1041,TJA1043,ATA6571 and other similar transceivers which have a go-to sleep and ERROR pin present.

1.8.3 Extended CDD CanTrcv

The RTA-BSW implementation of the CanTrcv module supports a fixed set of Transceivers (see the list in Supported Features), but support for additional CanTrcv types can be implemented through integration code and generated entry points, with the configuration of the additional CanTrcv being done in static code. The steps required to implement an Extended CDD CanTrcv are as follows.

Step 1. Configure CanTrcvRbHardwareType to CDD.

Step 2. Configure the CanTrcvRbCddName.

Step 3. Implement the required Functions.

The default Function pointers are as follows:

- <CanTrcvRbCddName>_CanTrcvInit
- <CanTrcvRbCddName>_CanTrcvSetOpMode
- <CanTrcvRbCddName>_CanTrcvGetOpMode
- <CanTrcvRbCddName>_CanTrcvGetBusWuReason
- <CanTrcvRbCddName>_CanTrcvSetWakeupMode
- <CanTrcvRbCddName>_CanTrcvCheckWakeup

If CanTrcvHwPnSupport is configured to TRUE for the CanTrcvChannel, the

following Function pointers will also be generated:

- <CanTrcvRbCddName>_CanTrcvCheckWakeFlag
- <CanTrcvRbCddName>_CanTrcvSetPNActivationState
- <CanTrcvRbCddName>_CanTrcvReadTrcvTimeoutFlag
- <CanTrcvRbCddName>_CanTrcvClearTrcvTimeoutFlag
- <CanTrcvRbCddName>_CanTrcvReadTrcvSilenceFlag
- <CanTrcvRbCddName>_CanTrcvClearTrcvWufFlag

The above Functions must be implemented in the CanTrcv/integration folder, and they must be declared in <CanTrcvRbCddName>.h header files, which are included in the CanTrcv_Cfg.h header file.

The full list of expected functions to be implemented for CDD CanTrcvChannels can be found in CanTrcv_CDDFctP_tst structure definition in CanTrcv_Cfg.c

- A sample implementation for a Dio CDD transceiver is available in CanTrcv/integration/CanTrcv_rba_TJA1041.c and CanTrcv/integration/CanTrcv_rba_TJA1041.h
- A sample implementation for a Spi CDD transceiver is available in CanTrcv/integration/CanTrcv_rba_TLE9255.c and CanTrcv/integration/CanTrcv_rba_TLE9255.h

Note that by default these files are not compiled due to the "#if 0" preprocessor definition at the top of each file.

Step 4. Implement your own code to control and obtain information from the CanTrcv hardware.

Please see file CanTrcv_Prv.h (section `/*Macros*/`) for detail about the mapping between parameter configurations and generated code

For example:

```
#define CANTRCV_INITSTATE(CANTRCVID)  CanTrcv_ConfigPtr->CanTrcvChannel[CANTRCVID].CanTrcvInitState_en
```

This maps `CanTrcvInitState` to `CanTrcv_ConfigPtr->CanTrcvChannel[CANTRCVID].CanTrcvInitState_en`.

1.8.3.1 Additional Parameters for CDD CanTrcv

The following parameters are required for an Extended CDD CanTrcv:

CanTrcvRbCddInitApiUsed

enable if a CDD Init API is available for this channel

CanTrcvRbCddMainApiUsed

enable if a CDD Main function API is available for this channel

CanTrcvRbCddGetBusWakeUpReasonApiUsed

enable if a CDD Get Bus Wakeup reason API is available for this channel

CanTrcvRbCddSetWakeupModeApiUsed

enable if a CDD Set Wakeup mode API is available for this channel

CanTrcvRbCddCheckWakeupApiUsed

enable if a CDD Check Wakeup API is available for this channel

:The parameter CanTrcvRbCddName in the configuration must be updated with CanTrcv_rba_TLE9255 for SPI based Trcvs and CanTrcv_rba_TJA1041 for DIO based Trcvs.This configuration will not only include a header file in the CanTrcv_cfg.h but also the addresses of corresponding CDD APIs with the name←CanTrcvRbCddName›_‹API_name›in CanTrcv_cfg.c. The API declarations are part of template files provided with the project. With this approach, the integration can be enabled and implemented to provide the CDD APIs as per the requirment and needed functionality.

1.8.4 Wakeup By Polling

In order to configure the wakeup by polling feature, set parameter `CanTrcvWakeUpSupport` to the value CANTRCV_WAKEUP_BY_POLLING. You should check that the transceiver hardware type supports this wakeup type before enabling this feature.

1.8.5 Bus Error Detection

You should always set the configuration parameter `CanTrcvBusErrFlag` to FALSE, as bus error detection is not supported in this release.

1.8.6 Partial Networking

To enable this power-saving feature, set configuration parameter `CanTrcvHwPnSupport` to TRUE. If you set the parameter to FALSE, the following API functions will be disabled:

- `CanTrcv_ReadTrcvTimeoutFlag`
- `CanTrcv_ClearTrcvTimeoutFlag`
- `CanTrcv_ReadTrcvSilenceFlag`
- `CanTrcv_GetTrcvSystemData`
- `CanTrcv_ClearTrcvWufFlag`

1.8.7 Initialisation State

To configure the state of a transceiver after initialisation (through the API function `CanTrcv_Init`), set the configuration parameter `CanTrcvInitState` to one of the following states:

- CANTRCV_OP_MODE_SLEEP
- CANTRCV_OP_MODE_STANDBY

1.8.8 Post-Build Variation Points

- Post-Build Variation Points can be placed between DIO Based Transceivers, between DIO and CDD, and between SPI Based Transceivers.
- Post-Build Variation Points cannot be placed between DIO and SPI based Transceivers, nor between CDD and SPI Based Transceivers.
- When a Variation Point is placed between DIO and CDD Transceiver for the CanTrcvRbHardwareType parameter, and Wakeup is configured as TRUE, the configuration of Wakeupsignalisation is required by setting the Variation Point for the CanTrcvAccess container.

1.8.9 Post-Build Loadable

CanTrcv does not support Post-Build Loadable for any of the configuration parameters.

1.8.10 Pre-Compile Time

CanTrcv supports only Pre-Compile Time. Hence, during initialization for Pre-compile Time, NULL_PTR is expected.

1.9 Integration Advice

1.9.1 Init Functions

CanTrcv is initialized by a call to CanTrcv_Init.

1.9.2 DeInit Functions

No DeInit is required. Notification of a request for CanTrcv to sleep is sent to the upper layer (CanIf).

1.9.3 Scheduled Functions

CanTrcv_MainFunction should be periodically scheduled, typically for 10ms, for the processing of various requests and API functionalities e.g. OpMode switching requests.

Note: Wakeup should only be checked when the ECU is in sleep mode and should be done by direct call of CheckWakeup and check WakeupSource.

1.9.4 CANTRCV_GETTIME

The integration file CanTrcv_Cfg_Time.h should be modified to provide `CANTRCV_GETTIME` and an appropriate definition of `CanTrcv_Cfg_Tick_to`.

`CANTRCV_GETTIME` should provide a time value from a free-running counter. The period of the counter must be less than or equal to 1 microsecond. `CANTRCV_CFG_TIME_FACTOR` should be set to the number of counter ticks per microsecond.

1.9.5 SPI External Buffer Mode

As stated in the Limitations section, CanTrcv currently only works with the External Buffer mode of SPI (Internal Buffer mode is not currently supported). For this reason, SPI should be configured as External Buffer mode by setting SpiChannelBuffersAllowed (in Spi/SpiGeneral) to 1 (EB, External Buffer).

1.9.6 Extended CDD CanTrcv

For both the Dio/Spi CDD CanTrcvChannel, in order to work correctly, the callback/indication to CanIf must be implemented in the integration functions, as summarized here:

Integration function	Required callback to CanIf
<CanTrcvRbCddName>_CanTrcvSetOpMode	CanIf_TrcvModeIndication
<CanTrcvRbCddName>_CanTrcvCheckWakeFlag	CanIf_CheckTrcvWakeFlagIndication
<CanTrcvRbCddName>_CanTrcvClearTrcvWufFlag	CanIf_ClearTrcvWufFlagIndication

These callbacks must be called with the correct TransceiverId parameter

1.9.7 CANTRCV_RB_CHECKWAKEUP_ASYNC_MODE

For `CANTRCV_RB_CHECKWAKEUP_ASYNC_MODE` the CanTrcv_rba_TLE9255.h integration file will require modification: For the applicable definition of `CanTrcv_MainFunctionCheckWakeup_CDD` the `CANTRCV_RB_CHECKWAKEUP_ASYNC_MODE` should be set to `CANTRCV_CFG_CHECKWAKEUP_ASYNC`, then the `CANTRCV_CFG_CHECKWAKEUP_ASYNC` should be set to 1.

1.9.8 CanTrcv_MainFunctionCheckWakeup_CDD

For `CanTrcv_MainFunctionCheckWakeup_CDD` the CanTrcv_rba_TLE9255.c integration file will require modification: Exclude the call to `EcuM_EndCheckWakeup` in the function `CanTrcv_MainFunctionCheckWakeup_CDD`, in order to compile successfully.

1.9.9 CAN_TRCV_DEM_SETEVENTSTATUS

For `CAN_TRCV_DEM_SETEVENTSTATUS` the CanTrcv_rba_TLE9255.c integration file will require modification: Call to `CAN_TRCV_DEM_SETEVENTSTATUS` and set the guard status with code: *#if (CANTRCV_CFG_MODE_PROPOGATION_DEM_USED_CDD == STD_ON)*, in order to not provide optional interface `Dem_SetEventStatus`.

1.9.10 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the imple-

mentationof these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The CanTrcv module uses the following OS resources:

- CanTrcv_RESOURCE

1.10 Hardware Requirements

Table 1.36.CanTrcv Hardware Requirements

Support	Usage
FLASH	1174 bytes
RAM	20 bytes

Usage calculated based on a sample configuration.

2 Glossary

CanTrcv
AUTOSAR CAN Transceiver module

CanIf
AUTOSAR CAN Interface module

DIO
Digital Input Output (SPAL module)

EcuM
AUTOSAR ECU State Manager module

SPI Serial Peripheral Interface Module

SPAL
Standard Peripheral Abstraction Layer

WUF
Wakeup Frame